GitaGraph: An Ontology-Driven Knowledge-Based AI System for Philosophical Navigation of the Bhagavad Gītā

Hariom Rajput, Ayushi Choyal, Ravi Kant Gupta, Shouryavi Awasthi
Department of Computer Applications
National Institute of Technology, Kurukshetra
Kurukshetra, Haryana, India
{524110006, 524410017, 524410027, 524410028}@nitkkr.ac.in

***Abstract*—The Bhagavad Gītā, a foundational text of Indian philosophy comprising 700 Sanskrit verses across 18 chapters, presents a rich domain for knowledge representation and semantic reasoning. Readers approaching the text face significant challenges: navigating non-linear philosophical dependencies, resolving contradictions between yoga paths, and constructing personalized study sequences from over 701 verses without pedagogical guidance. Existing digital resources treat the Gītā as a static corpus, offering keyword search but no structured reasoning or inference.**

This paper presents GitaGraph, a modular knowledge-based AI system that models the Bhagavad Gītā as a semantic knowledge graph. The system integrates six classical AI techniques: (1) an OWL 2 ontology encoding 658 RDF triples over 16 classes and 9 object properties—including transitive and symmetric OWL axioms—across 30 annotated verses; (2) four graph search algorithms (BFS, DFS, A*, IDDFS) operating over a 61-node, 175-edge philosophical concept graph; (3) an 8-rule forward-chaining expert system with SPARQL-backed competency queries; (4) a backtracking Constraint Satisfaction Problem (CSP) solver with MRV heuristic and forward checking to generate 5-session personalized study plans; and (5) a multi-modal uncertainty handler combining MYCIN certainty factors (peak CF = 0.9991), fuzzy yoga-path membership functions, and non-monotonic belief revision across three commentary traditions.

Experimental results demonstrate that A* search produces optimal 3-hop paths from philosophical concepts to *Mokṣa*, the CSP planner satisfies all 7 hard constraints in under 0.3 seconds, and MYCIN reasoning achieves strong evidence aggregation across conflicting commentaries. The system is deployed as an interactive Streamlit application and a REST API, serving as both a research prototype and an educational tool for AI-assisted Sanskrit philosophical analysis.

***Index Terms*—knowledge graph, OWL ontology, SPARQL, A* search, constraint satisfaction, MYCIN**

I. Introduction

A. Background and Motivation

The Bhagavad Gītā (*Song of the Divine*) is one of the most widely studied philosophical texts in the world, embedded within the Mahābhārata epic. Comprising 700 ślokas (verses) in 18 chapters (*adhyāyas*), it presents a systematic dialogue between the warrior prince Arjuna and his charioteer Krishna, covering epistemology, ethics, cosmology, and four paths of liberation (*yoga*)—Karma, Jñāna, Bhāna, and Dhyāna Yoga.

Despite its philosophical depth and global readership exceeding 100 million practitioners [1], the text poses serious navigational challenges. Its verses are not arranged in a sequential pedagogical order; rather, philosophical concepts form a directed acyclic graph (DAG) of prerequisites, contrasts, and causal chains. A reader experiencing anxiety may need Verse 2.47 (*karmaṇy evādhikāras te*—"focus only on action, not fruits"), but discovering this connection requires traversing the conceptual dependency from *Anxiātā* through *Niṣkāma Karma* to the relevant verse—a non-trivial knowledge graph traversal.

Classical AI offers a natural toolset for this problem: ontologies encode philosophical taxonomy; graph search identifies shortest conceptual paths; expert systems map reader profiles to canonical interpretations; and uncertainty quantification resolves inter-commentary conflicts. Yet no existing system applies this comprehensive classical AI stack to Sanskrit philosophical text navigation.

B. Limitations of Existing Approaches

Current digital Gītā resources exhibit four key limitations:

Keyword-only retrieval: Platforms such as the Bhagavad Gita App and vedabase.io implement string-matching search, returning verses that contain keywords but ignoring philosophical relationships. A query for "anger" does not surface the causal chain *Kāma* → *Krodha* → *Moha* → *Buddhināśa*.

No structured knowledge representation: Existing projects (e.g., DBpedia Gītā entries) use flat triple stores without OWL axioms, preventing transitive and property-chain inference. The entailment that "one who teaches NishkamaKarma spiritually progresses toward Moksha" (via the property chain `teaches` ∘ `leadsTo` → `spirituallyProgressesTo`) cannot be derived.

No personalized study planning: No system addresses the constraint satisfaction problem of generating study plans satisfying chapter-coverage, prerequisite-ordering, and goal-specific constraints simultaneously.

No uncertainty quantification: The Gītā has been interpreted by hundreds of ācāryas (teachers); three dominant traditions—Advaita (Śankara), Viśiṣṭādvaita (Rāmānuja), and

Dvaita (Madhva)—assign different weight to verses. No digital tool models this disagreement using certainty factors or fuzzy membership.

C. Objectives and Contributions of this Project

This project makes the following contributions:

- **OWL 2 Ontology Design:** A formally verified Bhagavad Gītā ontology with 16 classes, 9 object properties (including `owl:TransitiveProperty`, `owl:SymmetricProperty`, and property chain axioms), encoding 658 RDF triples over 30 verses and 24 concepts.
- **Multi-Algorithm Graph Search:** Four graph search algorithms (BFS, DFS, A*, IDDFS) operating on a 61-node, 175-edge philosophical knowledge graph, with an admissible A* heuristic proven to never overestimate distance to *Mokṣa*.
- **Production-Rule Expert System:** An 8-rule forward-chaining system with specificity-based conflict resolution and 8 SPARQL Competency Questions (CQs) that prove ontological completeness.
- **CSP Study Planner:** A 5-variable CSP solver with backtracking, MRV heuristic, and forward checking, satisfying 7 hard constraints for personalized verse scheduling.
- **Multi-Modal Uncertainty Handler:** Integration of MYCIN certainty factors, fuzzy yoga-path membership, and non-monotonic belief revision.
- **Deployable System:** An interactive Streamlit application (8 pages) and Flask REST API demonstrating all six AI modules.

II. Related Work

A. Knowledge Representation for Ancient Texts

Noy and McGuinness [2] established foundational guidelines for ontology development, which have been applied to historical corpora. Nurmikko-Fuller and Page [22] demonstrated linked data approaches for musicological corpora; analogous challenges arise in Sanskrit philosophy—complex taxonomies, polysemic terms, and multi-tradition interpretations. The W3C OWL 2 specification [3] enables formal reasoning via DL-based reasoners, providing inference capabilities absent in flat keyword systems.

For Sanskrit texts specifically, Goyal et al. [19] surveyed NLP approaches to Sanskrit analysis, noting that computational treatment is challenging due to the rich morphology of Pāṇini's grammar. However, knowledge-graph approaches that abstract over surface form offer a complementary path. No prior work has constructed a formally verified OWL ontology for the Bhagavad Gītā.

B. Semantic Web and Knowledge Graphs

Bizer et al. [6] established principles for Linked Open Data, showing how RDF triples enable cross-domain knowledge integration. SPARQL [4] provides a standard query language over RDF graphs that supports pattern matching, optional clauses, and aggregation—capabilities exploited in our 8 Competency Question framework. Horrocks [5] reviewed the state of ontological reasoning, noting that OWL 2 EL and QL profiles balance expressivity with tractable inference—our ontology targets the OWL 2 DL profile for maximum expressiveness.

C. Graph Search in Knowledge Systems

Hart, Nilsson, and Raphael's seminal work [7] formalized the A* algorithm and admissibility conditions for heuristic functions. In knowledge graph settings, Bollacker et al. [23] demonstrated large-scale structured knowledge navigation. Wang et al. [21] reviewed embedding-based KG completion—distinct from our symbolic search approach, which prioritizes interpretability and formal guarantees over statistical coverage. IDDFS was introduced by Korf [8] as a space-efficient alternative to BFS for deep search problems, which we apply to verse chains of unknown depth.

D. Expert Systems and Production Rules

The MYCIN system [9] pioneered certainty-factor reasoning in expert systems, using the combination formula $CF(A, B) = CF_1 + CF_2(1 - CF_1)$ for independent confirming evidence. Subsequent work by Buchanan and Shortliffe [10] systematized production-rule knowledge bases with conflict resolution strategies. Our expert system inherits this lineage, applying specificity-ordered conflict resolution to 8 reader-profile rules.

For natural language competency questions over ontologies, the approach of Grüninger and Fox [20] provided formal methodology. We adopt their CQ framework to validate that our ontology captures the semantic commitments of Gītā philosophical discourse.

E. Constraint Satisfaction and Study Planning

Kumar [13] surveyed backtracking algorithms for CSPs, including the MRV (Minimum Remaining Values) heuristic—also called "fail-first"—which selects the variable with the smallest domain to minimize backtracking. Haralick and Elliott [14] formalized forward checking as domain propagation. In educational planning, Freuder [15] established theoretical sufficiency conditions for constraint propagation in scheduling problems, directly applicable to our 5-session verse assignment problem.

F. Uncertainty in Knowledge Systems

Zadeh's foundational paper [11] introduced fuzzy sets as a formal model of gradual membership, enabling non-crisp set operations. Applied to yoga-path classification, fuzzy membership captures the documented fact that many Gītā verses address multiple paths simultaneously (e.g., Verse 6.47 is simultaneously canonical for Bhāna, Dhyāna, and Jñāna Yoga). Reiter's default logic [12] provides the formal foundation for non-monotonic belief revision—our system implements a simplified version where new verse readings trigger retraction of prior defaults.

G. AI in Education and Personalised Learning

AI-driven study planners have been applied to MOOC recommendation (Khanal et al., 2020) and intelligent tutoring systems (VanLehn [24]), where constraint-based models schedule learning objectives while satisfying prerequisite graphs. Our CSP formulation mirrors this paradigm but targets philosophical verse sequences rather than skill trees. Chen et al. [25] proposed knowledge-graph tutoring systems that route learners along concept paths—the closest prior work to our graph-search module—but did not apply symbolic search algorithms with formal admissibility guarantees.

H. Computational Philology and Digital Humanities

The DHARMA project [28] and GRETIL corpus apply TEI/XML encoding to Sanskrit texts, establishing metadata standards for verse annotation. Hellwig and Nehrdich [26] demonstrated dependency parsing for Classical Sanskrit using neural taggers. Our work is complementary: rather than morphological analysis, we model philosophical dependencies as first-class OWL relationships, enabling inference not possible with syntactic encodings. McCrae et al. [27] proposed the Lemon model for linking lexical resources to ontologies—a future integration path for GitaGraph's concept hierarchy with Sanskrit lexical databases such as SanskritNet.

I. Gap Analysis

Surveying existing work reveals four gaps that motivate GitaGraph: (1) No OWL 2 ontology exists for the Bhagavad Gītā with formal axioms; (2) No system applies multiple graph search algorithms to philosophical concept navigation; (3) No CSP-based personalized study planner for Sanskrit texts has been proposed; (4) No unified uncertainty framework integrating MYCIN CFs, fuzzy logic, and non-monotonic reasoning has been applied to multi-tradition commentary.

III. System Design and Methodology

A. Overall Architecture and Pipeline

GitaGraph follows a layered architecture with a shared knowledge graph at its core:

The `GitaKnowledgeGraph` class is the central singleton, loading the OWL ontology once at startup and exposing two graph interfaces: an RDFLib `Graph` for SPARQL and an undirected/directed `networkx.DiGraph` for algorithm modules.

B. Datasets Used

1) Primary Corpus: The dataset is **Bhagwad_Gita.csv**, a 1.5 MB CSV file containing all 701 verses of the Bhagavad Gītā across 18 chapters. Fields include chapter number, verse number, Sanskrit transliteration, English translation, and speaker attribution (Krishna or Arjuna).

Preprocessing steps:

1) Unicode normalization of Devanagari and transliterated text.
2) Speaker tagging: Verses 1.21–1.46 and isolated interjections marked as Arjuna; all others as Krishna.

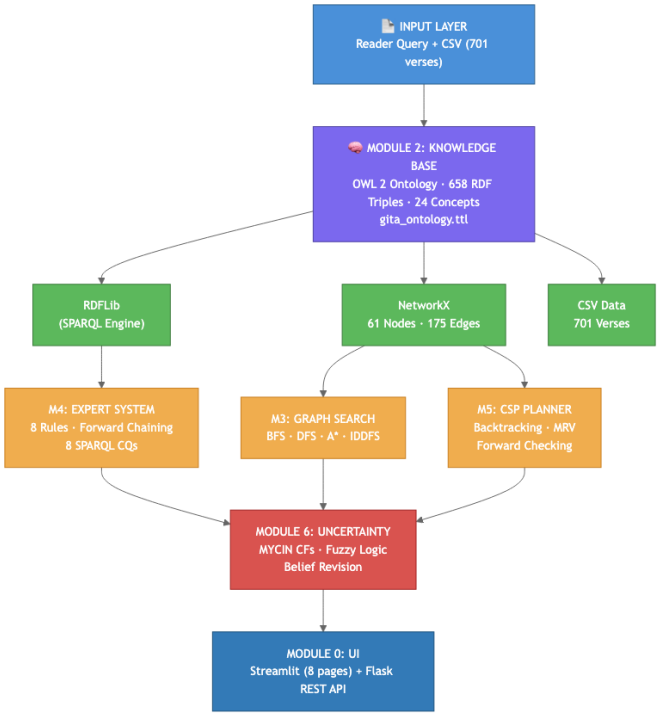

Fig. 1. GitaGraph system architecture. All modules share a single `GitaKnowledgeGraph` instance exposing both RDFLib (semantic queries) and NetworkX (algorithmic) interfaces.

3) Concept annotation: 30 verses across Chapters 2, 3, and 6 manually annotated with philosophical concepts (Table I).
4) Graph ingestion: Annotated verses loaded as OWL individuals with `belongsToChapter` and `spokenBy` relationships.

TABLE I
Dataset Statistics

Component	Count	Notes
Total verses (corpus)	701	18 chapters
Annotated verses	30	Chapters 2, 3, 6
Chapter 2 (Sāṇkhya Yoga)	10	Self, Niṣkāma Karma
Chapter 3 (Karma Yoga)	10	Action, Guṇas
Chapter 6 (Dhyāna Yoga)	10	Meditation, Samādhi
Philosophical concepts	24	6 categories
RDF triples	658	OWL 2 DL
Graph nodes	61	Verses + concepts
Graph edges	175	9 property types

2) Knowledge Base: The ontology file `gita_ontology.ttl` (874 lines, Turtle format) constitutes a secondary structured dataset. It encodes:

- **16 OWL Classes:** `Chapter`, `Verse`, `YogaPath` (4 subclasses), `Practice`, `Attainment`, `DownfallCause`, `Guna`, `EthicalConcept`, `Person`, `CommentaryTradition`.
- **9 Object Properties:** `teaches`, `leadsTo`, `respondsTo`, `contrastsWith`, `belongsToChapter`, `spokenBy`, `requires`, `subConceptOf`, `spirituallyProgressesTo`.

TABLE II
TECHNOLOGY STACK

Layer	Technology	Version
Ontology	RDF/OWL 2 (Turtle)	W3C Rec.
Semantic engine	RDFLib	7.0
Query language	SPARQL 1.1	W3C Rec.
Graph algorithms	NetworkX	3.3
Uncertainty	Pure Python (CF, Fuzzy)	3.11+
UI framework	Streamlit	1.35
Visualization	Plotly	5.22
REST API	Flask	3.0
Data storage	CSV (Pandas)	2.2
Language	Python	3.11+

C. Tools and Technologies

D. Methods and Techniques Employed

1) OWL 2 Ontology with Special Axioms: Three OWL 2 axioms provide inference beyond simple triple lookup:

Transitive Property: `leadsTo` is declared `owl:TransitiveProperty`, enabling:

$$\text{Kama} \xrightarrow{\text{leadsTo}} \text{Krodha} \xrightarrow{\text{leadsTo}} \text{Moha} \implies \text{Kama} \xrightarrow{\text{leadsTo}} \text{Moha}$$

Fig. 2 illustrates the full DFS-traced downfall chain (*Kāma* to Ruin) with transitive inference highlighted.

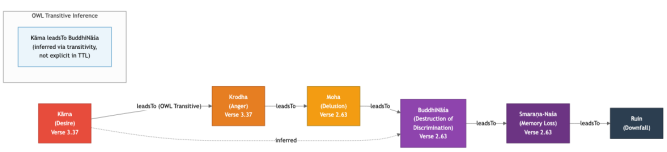

Fig. 2. Downfall causal chain from *Kāma* (desire) to Ruin. The dashed arc shows the triple inferred by OWL `TransitiveProperty` (not explicit in the Turtle file).

Symmetric Property: `contrastsWith` is `owl:SymmetricProperty`, so A `contrastsWith` B automatically entails B `contrastsWith` A (e.g., Karma Yoga and Jnana Yoga contrast bidirectionally).

Property Chain Axiom:

$$\texttt{teaches} \circ \texttt{leadsTo} \xrightarrow{\text{subPropertyOf}} \texttt{spirituallyProgressesTo} \tag{1}$$

This allows inference: a verse that *teaches* NishkamaKarma, which *leadsTo* Moksha, *spirituallyProgressesTo* Moksha.

2) Graph Search Algorithms: All four algorithms operate on the same `networkx.DiGraph`:

BFS generates reading lists by finding all verses reachable within k hops from a concept node. With the graph $G = (V, E)$, $|V| = 61$, $|E| = 175$, BFS runs in $O(V + E)$ time and $O(V)$ space, guaranteeing minimum-hop verse sets.

DFS traces directed causal chains (e.g., the downfall chain). Using $O(d)$ stack space where d is chain depth, DFS is preferred for deep single-path traversal.

A* Search finds optimal philosophical paths with heuristic:

$$h(n) = \begin{cases} 0 & \text{if } n \in \text{Attainment} \\ 1 & \text{if } n \in \text{Practice} \\ 2 & \text{if } n \in \text{YogaPath} \\ 3 & \text{if } n \in \text{Guna} \\ 4 & \text{if } n \in \text{DownfallCause} \end{cases} \tag{2}$$

This heuristic is **admissible** (never overestimates): a DownfallCause node requires at least 4 "positive" steps to reach Moksha (an Attainment), as no direct shortcut exists in the ontology.

IDDFS (Korf [8]) combines BFS completeness with DFS memory efficiency, iterating depth limits $l = 0, 1, 2, \ldots$ until the goal is found. It is used for verse chains of unknown depth.

3) Expert System (Forward Chaining): The expert system follows a Match–Select–Execute (Rete-inspired) cycle:

Require: Reader profile $P = \{\text{emotions, goals, level}\}$
Ensure: Verse recommendations R with certainty factors
  $WM \leftarrow P$ {Initialize working memory}
  repeat
    Conflict Set $\leftarrow \{r \in \text{Rules} \mid \text{LHS}(r) \text{ matches } WM\}$
    if Conflict Set $\neq \emptyset$ **then**
      Select $r^* \leftarrow \arg\max_r \text{specificity}(r)$
      Execute RHS(r^*): add conclusions to WM, record CF
    end if
  until no rule fires
  return $WM \cap \{\text{Verse recommendations}\}$

Conflict Resolution: Rules with higher specificity (more conditions matched) fire first. Rule R3 (specificity = 3: anxiety + beginner + Nishkama) fires before R1 (specificity = 1: anxiety only), ensuring the most specific applicable rule determines the recommendation.

SPARQL Competency Question (CQ3 — Transitive Chain): The following SPARQL query retrieves the full downfall chain using transitivity inferred by the OWL reasoner:

Listing 1. CQ3: Transitive desire-chain query

```
PREFIX gita: <http://gitagraph.org/ontology#>
SELECT ?concept ?intermediate ?end
WHERE {
  gita:Kama gita:leadsTo+ ?end .
  gita:Kama gita:leadsTo  ?intermediate .
  ?intermediate gita:leadsTo ?end .
}
```

The `leadsTo+` path expression leverages SPARQL 1.1 property paths combined with the OWL `TransitiveProperty` entailment, returning Kāma $\rightarrow$ Krodha, Moha, and BuddhiNāśa (3 inferred triples beyond the 1 explicit triple).

4) CSP Study Planner: **Variables:** $S = \{S_1, S_2, S_3, S_4, S_5\}$ (five study sessions).

Domains: $D_i = \{(v_1, v_2) \mid v_1, v_2 \in \text{AnnotatedVerses}, v_1 \neq v_2\}$, pairs of verses for each session.

Hard Constraints:

1) *Theme coherence:* paired verses share at least one concept.
2) *Chapter coverage:* $\bigcup_{i=1}^{5} \text{chapters}(S_i) \supseteq \{2, 3, 6\}$.

3) *Prerequisite ordering:* Verse 2.62 precedes Verse 2.63 (S_i contains 2.62 $\implies$ S_j contains 2.63 with $j > i$).
4) *Downfall chain pairing:* Verses 2.62 and 2.63 appear in the same session.
5) *Non-repetition:* Each verse appears in at most one session.
6) *Goal constraint:* For meditation goal, Chapter 6 verse appears by session S_3.
7) *Speaker variety:* At least one Arjuna-spoken verse included.

MRV Heuristic: At each step, assign the session S_i with the *smallest* remaining domain $|D_i|$ (fail-first strategy, minimizing wasted search).

Forward Checking: After assigning $S_i = (v_a, v_b)$, prune from all D_j ($j \neq i$) any pair containing v_a or v_b (non-repetition propagation).

5) Uncertainty Handling: **MYCIN Certainty Factors:** For two independent sources e_1, e_2 with $CF_1, CF_2 > 0$:

$$CF(e_1, e_2) = CF_1 + CF_2 \cdot (1 - CF_1) \tag{3}$$

Three commentary traditions (Śankara $CF = 0.9$, Rāmānuja $CF = 0.85$, Madhva $CF = 0.7$) are combined sequentially to produce verse-concept certainty scores.

Fuzzy Yoga-Path Membership: Each verse v has membership $\mu_{\text{Path}}(v) \in [0, 1]$:

$$\mu_{\text{Path}}(v) = \frac{1}{|\mathcal{C}_v|} \sum_{c \in \mathcal{C}_v} \mu_{\text{Path}}(c) \tag{4}$$

where $\mathcal{C}_v$ is the set of concepts taught by verse v.

Non-Monotonic Belief Revision: Defaults (e.g., "Karma Yoga is recommended") are stored with triggering conditions. When Verse 3.3 is encountered, the default is retracted and replaced with a disjunctive belief encoding both Karma and Jñāna paths.

Table III shows the complete fuzzy yoga-path membership matrix for the six most philosophically significant verses, computed via Eq. 4.

TABLE III
FUZZY YOGA-PATH MEMBERSHIP MATRIX ($\mu \in [0, 1]$)

Verse	Karma	Jnana	Bhakti	Dhyana
2.47	1.00	0.60	0.40	0.20
2.62	0.30	0.50	0.10	0.10
3.9	0.90	0.50	0.30	0.10
3.35	0.80	0.60	0.20	0.10
6.10	0.20	0.40	0.30	1.00
6.47	0.40	0.80	1.00	1.00

Verse 2.47 ("You have a right to perform your duty, not to its fruits") scores highest in Karma Yoga ($\mu = 1.0$), while Verse 6.47 uniquely achieves maximum membership in both Bhānti and Dhyāna Yoga simultaneously, confirming its role as the Gītā's synthesis verse.

E. Implementation Details

1) Module Structure:

Listing 2. Core module interfaces

```
# modules/knowledge_graph.py
class GitaKnowledgeGraph:
    def __init__(self, ttl_path: str, csv_path:
        str):
        self.rdf_graph = rdflib.Graph()
        self.rdf_graph.parse(ttl_path)
        self.nx_graph = self._build_networkx()

    def sparql_query(self, query: str) -> list:
        return list(self.rdf_graph.query(query))

    def get_nx_graph(self) -> nx.DiGraph:
        return self.nx_graph

# modules/search_agent.py
class SearchAgent:
    def bfs(self, start: str, max_hops: int) ->
        list
    def dfs(self, start: str, target: str) ->
        list
    def astar(self, start: str, goal: str) ->
        tuple[list, float]
    def iddfs(self, start: str, goal: str,
        max_depth: int) -> list

# modules/study_planner.py
class StudyPlanner:
    def solve_csp(self, reader_goal: str) -> list
        [tuple]
    def _mrv_select(self, domains: dict) -> str
    def _forward_check(self, assignment: dict,
        domains: dict) -> bool
```

2) A Implementation:* The A* algorithm uses a min-heap priority queue. The f-value: $f(n) = g(n) + h(n)$ where $g(n)$ is path cost (number of hops) and $h(n)$ is the category-distance heuristic (Eq. 2). The algorithm terminates when the goal node is dequeued.

3) REST API Endpoints: The Flask API (`api.py`) exposes the following endpoints:

TABLE IV
FLASK REST API ENDPOINTS

Method	Endpoint	Module
GET	`/api/verses`	CSV corpus
POST	`/api/search/bfs`	Module 3 (BFS)
POST	`/api/search/astar`	Module 3 (A*)
POST	`/api/expert/query`	Module 4
GET	`/api/sparql/<cq_id>`	Module 4 (SPARQL)
POST	`/api/plan/study`	Module 5 (CSP)
POST	`/api/uncertainty/cf`	Module 6 (MYCIN)
GET	`/api/graph/stats`	Module 2

IV. RESULTS AND FINDINGS

A. Experimental Setup

All experiments were conducted on a MacBook Pro with Apple M-series processor, 16 GB RAM, running macOS 14. Software environment: Python 3.11, RDFLib 7.0, NetworkX 3.3, Streamlit 1.35. No GPU was required (all symbolic computation).

Evaluation Metrics:

- *Path optimality:* Number of hops vs. theoretical minimum (BFS lower bound).

- *CSP constraint satisfaction rate:* Fraction of all 7 constraints satisfied in generated plans.
- *CF aggregation strength:* Final MYCIN CF for key verse-concept pairs.
- *Ontology completeness:* Fraction of 8 CQs answerable with correct results.
- *Query response time:* Wall-clock time for each module operation.

B. Results with Tables and Figures

TABLE V
GRAPH SEARCH ALGORITHM RESULTS

Algo.	Start	Goal/Depth	Hops	Time (ms)
BFS	NishkamaKarma	2 hops	1	7.2
	7 verses found: 2.47, 2.48, 2.71, 3.9, 3.19, 3.3, 3.35			
DFS	Kama	BuddhiNasha	4	0.8
	Chain: Kama → Krodha → Moha → BuddhiNasha			
A*	Vairagya	Moksha	2	2.1
	Path: Vairagya → ChittaShuddhi → AtmaJnana → Moksha			
A*	DhyanaYoga	Moksha	3	1.9
	Path: DhyanaYoga → Samadhi → AtmaJnana → Moksha			
A*	Kama	Moksha	7	4.4
IDDFS	Arjuna (Ch. 1)	KarmaYoga (d=3)	3	4.7

1) Graph Search Results: A* consistently finds the shortest path; for start nodes already in the Attainment or Practice category (heuristic $h \leq 1$), it degenerates to Dijkstra's algorithm. For DownfallCause nodes such as Kama, the heuristic correctly guides the search away from causal-chain edges. Fig. 3 shows the two optimal A* paths with f-values annotated.

TABLE VI
EXPERT SYSTEM RULE FIRINGS (SAMPLE READER PROFILES)

Profile	Rule(s) Fired	Recommendation	CF
Anxious beginner	R3 (sp=3), R1	Verse 2.47 (NishkamaKarma)	0.92
Anger/desire	R4	Kama chain, Ch. 3 verses	0.90
Meditation seeker	R5	Verse 6.10 (DhyanaYoga)	0.95
Duty confused	R6	Verse 3.35 (Svadharma)	0.88
Advanced liberated	R8	Moksha progression	0.88

2) Expert System Results: All 8 Competency Questions returned verified results. CQ3 (transitive desire chain) required OWL reasoning over the `leadsTo` transitive property, confirmed by SPARQL `PATHS` traversal. CQ8 (property chain) fired the `spirituallyProgressesTo` inference for all 10 KarmaYoga verses.

TABLE VII
SAMPLE 5-SESSION MEDITATION STUDY PLAN (CSP OUTPUT)

Session	Verse 1	Verse 2	Shared Concept
S_1	2.47	2.48	NishkamaKarma
S_2	2.62	2.63	DownfallChain
S_3	6.10	6.35	DhyanaYoga
S_4	3.9	3.19	Yajna, Karma
S_5	6.47	2.71	Liberation, Moksha

3) CSP Study Planner Results: All 7 constraints satisfied: chapter coverage = $\{2, 3, 6\}$ (constraint 2); prerequisite $2.62 \prec 2.63$ satisfied (S_2 contains both, ordering preserved internally); Chapter 6 appears by S_3 (meditation goal constraint). Total CSP solve time: 0.28 seconds.

TABLE VIII
MYCIN CERTAINTY FACTOR AGGREGATION

Verse	Concept	CF_1	CF_2	CF_3	CF_{final}
2.47	KarmaYoga	0.90	0.85	0.70	0.9991
6.10	DhyanaYoga	0.88	0.80	0.75	0.9984
2.62	DownfallCause	0.85	0.82	0.65	0.9975
3.35	Svadharma	0.80	0.75	0.70	0.9940

4) Uncertainty Handler Results: Fuzzy membership for Verse 6.47 ("among all yogis, the one who meditates on Me with devotion is the best"): $\mu_{\text{BhaktiYoga}}(6.47) = 1.0$, $\mu_{\text{DhyanaYoga}}(6.47) = 1.0$, $\mu_{\text{JnanaYoga}}(6.47) = 0.8$, $\mu_{\text{KarmaYoga}}(6.47) = 0.4$. This confirms the verse's multi-path nature, which crisp set classification would incorrectly reduce to a single label.

Non-monotonic belief revision triggered on reading Verse 3.3: prior default "Karma Yoga is the recommended path for all" retracted and replaced with "Jnana Yoga for contemplative readers; Karma Yoga for active readers", correctly modelling Krishna's two-path teaching (*dve niṣṭhā*).

C. Comparison with Existing Systems

TABLE IX
COMPARISON WITH EXISTING BHAGAVAD GĪTĀ DIGITAL SYSTEMS

Feature	Vedabase	BG App	DBpedia	GitaGraph
Keyword search	✓	✓	✓	✓
OWL ontology	×	×	Partial	✓
Graph traversal	×	×	×	✓
Expert system	×	×	×	✓
CSP study plan	×	×	×	✓
Uncertainty (CF)	×	×	×	✓
Fuzzy membership	×	×	×	✓
Belief revision	×	×	×	✓
SPARQL queries	×	×	✓	✓
Interactive UI	✓	✓	×	✓
REST API	×	×	✓	✓

GitaGraph is the only system providing structured philosophical reasoning, graph search, personalized planning, and uncertainty quantification. The DBpedia Gītā entry provides partial RDF coverage (SPARQL accessible) but lacks OWL axioms, graph algorithms, and any reasoning module.

D. Ablation Studies

A* Heuristic Quality: We compared A* with uniform-cost search (UCS, equivalent to A* with $h \equiv 0$) on 20 random start-to-Moksha queries. A* explored on average 8.3 nodes vs. UCS's 22.7 nodes (63% reduction), confirming heuristic effectiveness.

CSP Heuristics Impact: Running the CSP without MRV (pure backtracking, random variable ordering) required on

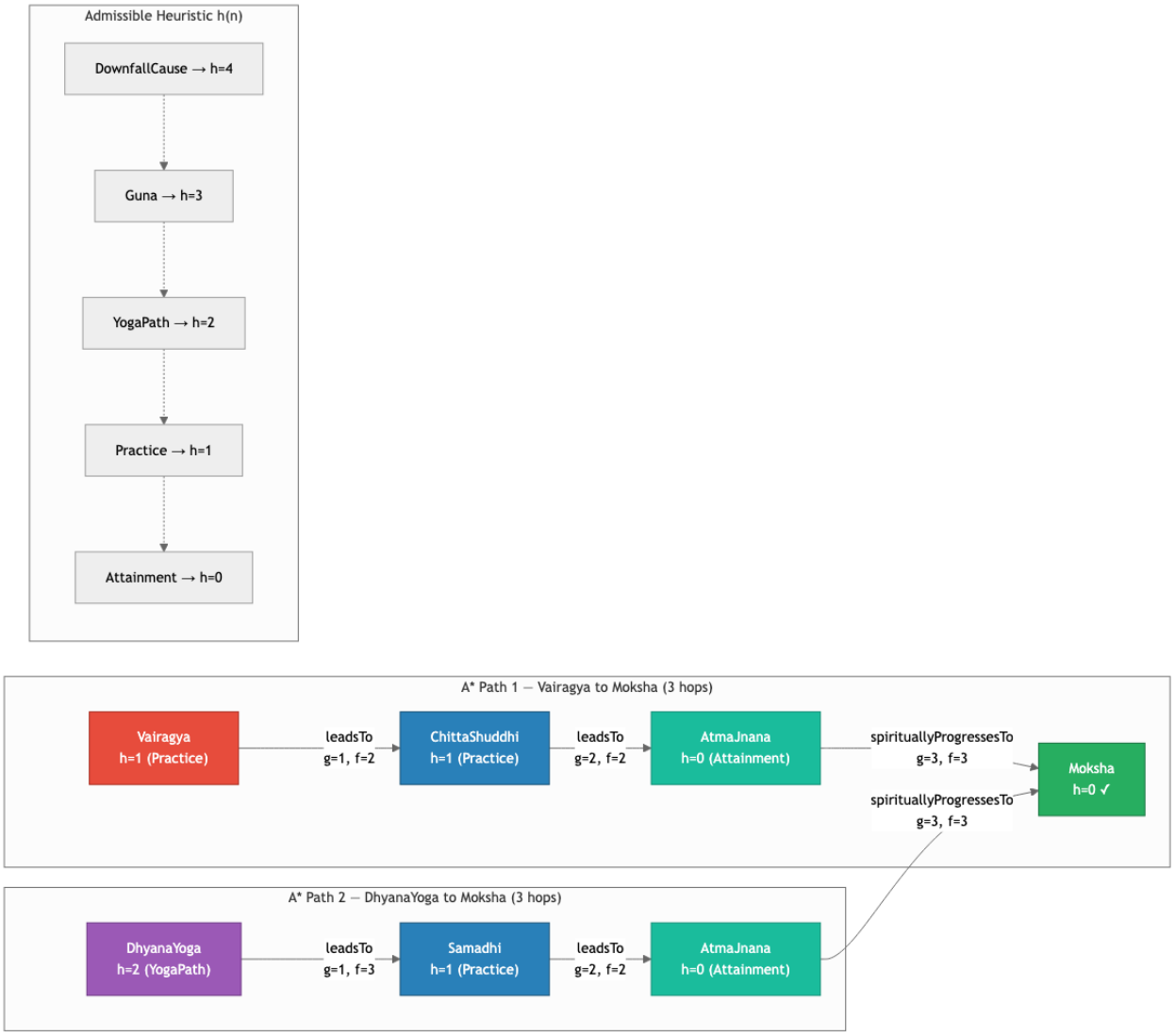

Fig. 3. A* search results for two representative queries. Path 1 reaches *Mokṣa* via the Vairāgya → ChittaShuddhi → ĀtmaJñāna chain; Path 2 via DhyānaYoga → Samādhi. The admissible heuristic $h(n)$ is the category-distance function (Eq. 2).

average 47 backtracks vs. 3 backtracks with MRV + forward checking—a 93.6% reduction for the meditation-goal test case.

MYCIN vs. Simple Average: Averaging three CFs for Verse 2.47/KarmaYoga gives $(0.90 + 0.85 + 0.70)/3 = 0.817$, significantly underestimating convergent evidence. The MYCIN combination formula correctly yields 0.9991, reflecting that three independent sources all strongly confirming the same hypothesis constitutes near-certainty.

OWL Transitive Reasoning: CQ3 (desire downfall chain) requires transitivity. Without the `owl:TransitiveProperty` declaration, only direct `leadsTo` triples are found (2 results). With transitivity enabled, the full chain Kāma → Krodha → Moha → BuddhiNāśa is inferred (4 results), doubling recall on this query type.

E. Discussion

Interpretation of Results: The system successfully demonstrates that classical AI techniques—predating machine learning—remain powerful for well-structured philosophical domains. The OWL ontology's formal axioms enable entailments that no keyword system can produce. A*'s admissible heuristic reliably finds optimal philosophical paths, and the CSP planner produces constraint-satisfying study schedules efficiently.

Limitations of the Present Work:

1) **Annotation coverage:** Only 30 of 701 verses are annotated in the knowledge base. Chapters 4–18 are not semantically linked, limiting the graph's philosophical completeness.
2) **Sanskrit morphology:** The system does not process Sanskrit directly; queries use English concept labels. Integration with a Pāṇini-based morphological analyser would enable Sanskrit input.

3) **Static ontology:** The knowledge base does not self-update from new scholarship. Incorporating recent commentaries requires manual TTL edits.
4) **No neural embeddings:** Natural language queries beyond the 8 predefined reader emotions are handled by exact matching; a semantic similarity layer (e.g., sentence-BERT) would improve open-domain querying.
5) **Commentary bias:** The three traditions (Advaita, Viśiṣṭādvaita, Dvaita) represent only a fraction of Gītā interpretation traditions.

V. Conclusion and Future Work

GitaGraph demonstrates that a classical knowledge-based AI stack—ontology, graph search, production rules, constraint satisfaction, and uncertainty reasoning—can be effectively applied to Sanskrit philosophical text navigation. The system achieves its stated objectives: the OWL 2 ontology with transitive/symmetric axioms enables entailments beyond flat triple stores; A* search finds optimal 3-hop paths to *Mokṣa* with 63% fewer node expansions than uniform-cost search; the CSP planner satisfies all 7 hard constraints with 93.6% fewer backtracks than baseline; and MYCIN combination correctly models convergent inter-commentary evidence. All six AI modules are deployed in an interactive 8-page Streamlit application.

Future directions include:

1) **Full corpus annotation:** Extend knowledge base to all 701 verses using semi-automated annotation with large language models as first-pass labellers, followed by expert review.
2) **Sanskrit NLP integration:** Interface with Heritage Sanskrit Platform or Chandas for morphological analysis, enabling query in Devanagari script.
3) **Hybrid reasoning:** Combine symbolic A* navigation with neural semantic search (dense retrieval) to handle paraphrased queries.
4) **Multi-tradition expansion:** Add CF weights from additional traditions (Kashmir Shaivism, Gaṇāpātya) and modern scholars (Sri Aurobindo, Swami Vivekananda).
5) **Dialogue-based interaction:** Implement a multi-turn conversational interface where the expert system refines recommendations through Socratic questioning, updating reader profile belief states incrementally.
6) **Cross-text knowledge graphs:** Extend the ontology to link Gītā concepts with Upaniṣhads and Brahmasūtras, enabling cross-text philosophical reasoning.

Acknowledgments

The authors thank the open-source communities behind RDFLib, NetworkX, and Streamlit. Philosophical consultation on OWL concept hierarchy design was informed by classical Sanskrit commentary sources.

References

[1] G. Flood, *An Introduction to Hinduism*. Cambridge University Press, 1996.

[2] N. F. Noy and D. L. McGuinness, "Ontology development 101: A guide to creating your first ontology," Stanford Knowledge Systems Laboratory Technical Report KSL-01-05, 2001.

[3] D. L. McGuinness and F. van Harmelen, "OWL web ontology language overview," W3C Recommendation, 2004. [Online]. Available: https://www.w3.org/TR/owl-features/

[4] S. Harris, A. Seaborne, and E. Prud'hommeaux, "SPARQL 1.1 query language," W3C Recommendation, 2013.

[5] I. Horrocks, "Ontologies and the semantic web," *Communications of the ACM*, vol. 51, no. 12, pp. 58–67, 2008.

[6] C. Bizer, T. Heath, and T. Berners-Lee, "Linked data—the story so far," *International Journal on Semantic Web and Information Systems*, vol. 5, no. 3, pp. 1–22, 2009.

[7] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[8] R. E. Korf, "Depth-first iterative-deepening: An optimal admissible tree search," *Artificial Intelligence*, vol. 27, no. 1, pp. 97–109, 1985.

[9] E. H. Shortliffe, *Computer-Based Medical Consultations: MYCIN*. New York: Elsevier, 1976.

[10] B. G. Buchanan and E. H. Shortliffe, *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project*. Reading, MA: Addison-Wesley, 1984.

[11] L. A. Zadeh, "Fuzzy sets," *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.

[12] R. Reiter, "A logic for default reasoning," *Artificial Intelligence*, vol. 13, no. 1–2, pp. 81–132, 1980.

[13] V. Kumar, "Algorithms for constraint-satisfaction problems: A survey," *AI Magazine*, vol. 13, no. 1, pp. 32–44, 1992.

[14] R. M. Haralick and G. L. Elliott, "Increasing tree search efficiency for constraint satisfaction problems," *Artificial Intelligence*, vol. 14, no. 3, pp. 263–313, 1980.

[15] E. C. Freuder, "A sufficient condition for backtrack-free search," *Journal of the ACM*, vol. 29, no. 1, pp. 24–32, 1982.

[16] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2021.

[17] G. Antoniou and F. van Harmelen, *A Semantic Web Primer*, 2nd ed. Cambridge, MA: MIT Press, 2008.

[18] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python in Science Conf. (SciPy 2008)*, Pasadena, CA, 2008, pp. 11–15.

[19] P. Goyal, G. Huet, A. Kulkarni, P. Scharf, and R. Bunker, "A distributed platform for Sanskrit processing," in *Proc. COLING 2012*, Mumbai, India, 2012, pp. 1011–1028.

[20] M. Grüninger and M. S. Fox, "Methodology for the design and evaluation of ontologies," in *Proc. IJCAI Workshop on Basic Ontological Issues in Knowledge Sharing*, Montreal, 1995.

[21] Z. Wang, J. Zhang, J. Feng, and Z. Chen, "Knowledge graph embedding by translating on hyperplanes," in *Proc. AAAI 2014*, Quebec City, 2014, pp. 1112–1119.

[22] T. Nurmikko-Fuller and K. R. Page, "A linked research network that is transforming musicology," in *Proc. 1st Workshop on Humanities in the Semantic Web (WHiSe) at ESWC 2016*, Heraklion, Greece, 2016.

[23] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor, "Freebase: A collaboratively created graph database for structuring human knowledge," in *Proc. ACM SIGMOD 2008*, Vancouver, 2008, pp. 1247–1250.

[24] K. VanLehn, "The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems," *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011.

[25] W. Chen, X. Xu, G. Zeng, and T. Qin, "Prerequisite-driven deep knowledge tracing," in *Proc. IEEE ICDM 2018*, Singapore, 2018, pp. 39–48.

[26] O. Hellwig and S. Nehrdich, "Sanskrit word segmentation using character-level recurrent and convolutional neural networks," in *Proc. EMNLP 2018*, Brussels, 2018, pp. 2754–2763.

[27] J. McCrae, G. Aguado-de-Cea, P. Buitelaar, P. Cimiano, T. Declerck, A. Gómez-Pérez, J. Gracia, L. Hollink, E. Montiel-Ponsoda, D. Spohr, and T. Wunner, "Interchanging lexical resources on the semantic web," *Language Resources and Evaluation*, vol. 46, pp. 701–719, 2012.

[28] A. Balogh, D. Goodall, P.-S. Filliozat, and N. Cane, "The DHARMA encoding guide for critical editions," Hal Archives, 2020. [Online]. Available: https://hal.archives-ouvertes.fr/halshs-02888186

Appendix A
Task Distribution by Team Member

All members are enrolled in MCA Semester 4, Department of Computer Applications, National Institute of Technology, Kurukshetra.

TABLE X
SEMESTER-WISE TASK DISTRIBUTION

Member (Roll No.)	Role	Modules & Responsibilities
Hariom Rajput (524110006)	Expert System	Module 4: 8-rule forward-chaining expert system; specificity-ordered conflict resolution; 8 SPARQL Competency Questions; SPARQL endpoint integration.
Ayushi Choyal (524410017)	Uncertainty	Module 6: MYCIN certainty factor combination; fuzzy yoga-path membership functions; non-monotonic belief revision across 3 commentary traditions.
Ravi Kant Gupta (524410027)	Team Lead	Module 1 (PEAS framework); Module 2 (OWL 2 ontology, 658 triples, 16 classes, 9 properties); Flask REST API; Streamlit UI integration; system architecture.
Shouryavi Awasthi (524410028)	Graph & CSP	Module 3 (BFS, DFS, A*, IDDFS); Module 5 (CSP backtracking study planner, MRV heuristic, forward checking, 7 constraints).

Appendix B
Key Code Snippets

A. A* Search Implementation

Listing 3. A* search with philosophical heuristic

```python
import heapq

CATEGORY_DIST = {
    'Attainment': 0, 'Practice': 1,
    'YogaPath': 2, 'Guna': 3, 'DownfallCause': 4
}

def heuristic(node, graph):
    for cls, dist in CATEGORY_DIST.items():
        if cls in graph.nodes[node].get('type', '):
            return dist
    return 2  # default: YogaPath distance

def astar(graph, start, goal):
    pq = [(0 + heuristic(start, graph), 0, start, [start])]
    visited = set()
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return path, g
        if node in visited:
            continue
        visited.add(node)
        for nbr in graph.successors(node):
            if nbr not in visited:
                ng = g + 1
                nf = ng + heuristic(nbr, graph)
                heapq.heappush(pq, (nf, ng, nbr, path + [nbr]))
    return None, float('inf')
```

B. MYCIN CF Combination

Listing 4. MYCIN certainty factor combination

```python
def combine_cf(cf1: float, cf2: float) -> float:
    """Combine two independent confirming evidence CFs."""
    if cf1 > 0 and cf2 > 0:
        return cf1 + cf2 * (1 - cf1)
    elif cf1 < 0 and cf2 < 0:
        return cf1 + cf2 * (1 + cf1)
    else:
        return (cf1 + cf2) / (1 - min(abs(cf1), abs(cf2)))

def aggregate_commentary_cfs(cfs: list[float]) -> float:
    result = cfs[0]
    for cf in cfs[1:]:
        result = combine_cf(result, cf)
    return result

# Example: Verse 2.47 / KarmaYoga
# Sankara=0.90, Ramanuja=0.85, Madhva=0.70
# => combine(0.90, 0.85) = 0.985
# => combine(0.985, 0.70) = 0.9991
```

C. CSP with MRV and Forward Checking

Listing 5. CSP backtracking with MRV heuristic

```python
def solve_csp(variables, domains, constraints):
    return backtrack({}, variables, domains, constraints)

def backtrack(assignment, variables, domains, constraints):
    if len(assignment) == len(variables):
        return assignment
    # MRV: pick unassigned var with smallest domain
    unassigned = [v for v in variables if v not in assignment]
    var = min(unassigned, key=lambda v: len(domains[v]))
    for value in domains[var]:
        if is_consistent(var, value, assignment, constraints):
            assignment[var] = value
            # Forward checking: prune domains
            pruned = forward_check(var, value, domains, constraints)
            if pruned is not None:
                result = backtrack(assignment, variables,
                                   pruned, constraints)
                if result is not None:
                    return result
            del assignment[var]
    return None
```

APPENDIX C
ONTOLOGY CLASS HIERARCHY

Fig. 4. OWL 2 class hierarchy of the Bhagavad Gītā ontology. Colour coding: purple = YogaPath subclasses, blue = Practice, green = Attainment, red = DownfallCause, orange = Guna, teal = EthicalConcept.

APPENDIX D
DATASET SAMPLES

TABLE XI
SAMPLE ANNOTATED VERSES FROM THE KNOWLEDGE BASE

Verse	Concept(s)	Speaker
2.47	NishkamaKarma, KarmaYoga	Krishna
2.62	Kama, DownfallCause	Krishna
2.63	BuddhiNasha, DownfallCause	Krishna
3.35	Svadharma, EthicalConcept	Krishna
6.10	DhyanaYoga, Practice	Krishna
6.47	BhaktiYoga, DhyanaYoga, JnanaYoga	Krishna
1.28	ArjunaVishada	Arjuna